DREAMSCAPE — Project Setup & Configuration Guide

AI Dream Journal & Surrealist Film Generator Gemini Live Agent Challenge — Creative Storyteller Category Last Updated: March 2026

Table of Contents

1. Project Overview
2. Hackathon Requirements Tracker
3. GCP Configuration — What's Done
4. GCP Configuration — What's Pending
5. Open Issues & Resolutions
6. Credentials & Config Reference
7. Gemini Interleaved Output Verification
8. Architecture Overview
9. Technology Stack
10. Agent Pipeline Design
11. Data Models
12. API Design
13. Frontend Design
14. Repository Structure
15. Development Roadmap
16. Demo Script (4-Minute Video)
17. Submission Checklist
18. Local Development Setup

---

1. Project Overview

Dreamscape transforms fragmented, messy dream descriptions into immersive multimedia experiences — surrealist short films combining AI-generated imagery, narrated voiceover, ambient audio, and dissolving text fragments, all produced through Gemini's native interleaved output capabilities.

Over time, it builds a visual dream journal and uses Gemini to identify recurring symbols, emotional arcs, and thematic patterns across dream entries.

Category: Creative Storyteller ✍️ **Core Requirement:** Gemini's interleaved/mixed output capabilities (text + images in single response stream)

2. Hackathon Requirements Tracker

Mandatory Requirements

#	Requirement	Implementation	Status
1	Leverage a Gemini model	Gemini 2.0 Flash (interleaved output)	✅ API key obtained
2	Google GenAI SDK or ADK	Google GenAI SDK (Python) + ADK for agent orchestration	☐ To build
3	At least one Google Cloud service	Cloud Run, Cloud Storage, Firestore, Cloud TTS	✅ Provisioned
4	Interleaved/mixed output	Gemini native text + image in single stream	⚠️ Needs verification
5	Hosted on Google Cloud	Backend on Cloud Run	☐ To deploy

Submission Deliverables

#	Deliverable	Status
1	Text description (features, tech, learnings)	☐ To write
2	Public GitHub repository URL	☐ To create
3	README with spin-up instructions	☐ To write
4	GCP deployment proof (screen recording)	☐ To record
5	Architecture diagram	☐ To create
6	Demo video (<4 minutes)	☐ To record

Bonus Points

#	Bonus Item	Implementation	Status
1	Blog/content with #GeminiLiveAgentChallenge	Blog post planned	☐ To publish
2	Automated Cloud Deployment (IaC)	Terraform in /infra	☐ To build
3	GDG membership + profile link	Sign up at developers.google.com	☐ To do

3. GCP Configuration — What's Done

3.1 Project

Item	Value
GCP Project ID	`dreamscape-hackathon`
Console URL	https://console.cloud.google.com/home/dashboard?project=dreamscape-hackathon

3.2 Enabled APIs

API	Status
Generative Language API (Gemini)	✅ Enabled
Cloud Run Admin API	✅ Enabled
Cloud Firestore API	✅ Enabled
Cloud Storage API	✅ Enabled
Artifact Registry API	✅ Enabled
Cloud Build API	✅ Enabled

3.3 Gemini / AI Studio

Item	Value
API Key	`AIzaSyC0QlN_yWtph0CnkwABggBaZFO9A7X7kzc`
Available Model	Gemini 2.0 Flash (confirmed image generation works)
Interleaved Output	⚠️ Needs verification test (see Section 7)
Access URL	https://aistudio.google.com

3.4 Firebase

Item	Value
Project	`dreamscape-hackathon` (linked to GCP project)

Item	Value
Auth Provider	Google Sign-In ✅ Enabled
Firebase Hosting	✅ Enabled
API Key	`AIzaSyD6qfA1PfrF5PZx7qSS0k00PYnn9vll034`
Auth Domain	`dreamscape-hackathon.firebaseapp.com`
Storage Bucket	`dreamscape-hackathon.firebasestorage.app`
Messaging Sender ID	`411432678431`
App ID	`1:411432678431:web:15a0c7d640aa7fcf97d12a`
Measurement ID	`G-JXMQE6ZBVY`

Firebase Config Block:

```javascript
const firebaseConfig = {
  apiKey: "AIzaSyD6qfA1PfrF5PZx7qSS0k00PYnn9vll034",
  authDomain: "dreamscape-hackathon.firebaseapp.com",
  projectId: "dreamscape-hackathon",
  storageBucket: "dreamscape-hackathon.firebasestorage.app",
  messagingSenderId: "411432678431",
  appId: "1:411432678431:web:15a0c7d640aa7fcf97d12a",
  measurementId: "G-JXMQE6ZBVY"
};
```

3.5 IAM Service Account

Item	Value
Service Account	`backend@dreamscape-hackathon.iam.gserviceaccount.com`
Key Download	⛔ Blocked by org policy — **NOT NEEDED** (using Application Default Credentials instead)

3.6 Cloud Storage

Item	Value
Bucket Name	`dreamscape-media`
Purpose	Generated images, audio, video files

3.7 Firestore

Item	Value
Database	Default database created
Mode	Native mode (confirm this — Datastore mode cannot be changed later)

---

4. GCP Configuration — What's Pending

4.1 APIs Still Needed

API	Why Needed	How to Enable
Cloud Text-to-Speech API	Narration voiceover generation	Console → APIs & Services → Enable → search "Cloud Text-to-Speech API"
Secret Manager API	Secure storage of API keys in production	Console → APIs & Services → Enable → search "Secret Manager API"
Vertex AI API (optional)	Alternative route for Imagen 3 if needed	Console → APIs & Services → Enable → search "Vertex AI API"

4.2 IAM Roles Still Needed on Service Account

Grant these roles to `backend@dreamscape-hackathon.iam.gserviceaccount.com` (no key file needed — these roles are used when Cloud Run runs under this service account):

Role	Role ID	Why
Cloud Run Invoker	`roles/run.invoker`	Backend invocation
Firestore User	`roles/datastore.user`	Read/write dream data
Storage Object Admin	`roles/storage.objectAdmin`	Upload/manage media files
Cloud Text-to-Speech User	`roles/cloudtts.user`	⚠️ Enable TTS API first (see below)
Secret Manager Secret Accessor	`roles/secretmanager.secretAccessor`	Access API keys at runtime

How to grant roles via CLI (recommended):

```bash
```

```
SA=backend@dreamscape-hackathon.iam.gserviceaccount.com
PROJECT=dreamscape-hackathon

gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA" --role="roles/run.invoker"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA" --role="roles/datastore.user"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA" --role="roles/storage.objectAdmin"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA" --role="roles/secretmanager.secretAcc
# Run this AFTER enabling Cloud Text-to-Speech API:
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA" --role="roles/cloudtts.user"
```

Note on Text-to-Speech role: This role won't appear in the Console dropdown until you enable the Cloud Text-to-Speech API. Enable the API first (see Section 4.1), then grant the role using the CLI command above.

4.3 Artifact Registry Repository

Create a Docker repository:

```bash
gcloud artifacts repositories create dreamscape-repo \
  --repository-format=docker \
  --location=us-central1 \
  --project=dreamscape-hackathon
```

---

5. Open Issues & Resolutions

Issue 1: Service Account Key — RESOLVED ✅

Problem: `iam.disableServiceAccountKeyCreation` org constraint is **Enforced** at the organization level (inherited by the project). The policy source is "Inherit parent's policy", meaning it cannot be overridden at the project level even by the project owner.

Resolution: No key file needed. Using Application Default Credentials instead.

This is actually the **recommended Google practice** — more secure than downloading a JSON key file.

For Local Development:

```bash
# Authenticate as yourself — all Google Cloud SDKs will use your credentials
gcloud auth application-default login
```

This makes Firestore, Cloud Storage, Cloud TTS, and all other Google Cloud client libraries work locally without any service account key file.

For Cloud Run (Production): Attach the service account directly at deploy time — Cloud Run auto-injects credentials:

```bash
gcloud run deploy dreamscape-backend \
  --service-account=backend@dreamscape-hackathon.iam.gserviceaccount.com \
  --project=dreamscape-hackathon
```

No key file involved. Cloud Run handles authentication automatically.

For Gemini API: The AI Studio API key (`GOOGLE_API_KEY`) is a separate thing entirely — it's passed as an environment variable and does not use service account auth.

Bottom line: The org policy is a non-issue. We proceed without a key file.

Issue 2: Verify Firestore Mode

Action needed: Confirm your Firestore is in **Native mode**, not Datastore mode.

Check: Go to Console → Firestore. If you see a "Collections" panel, you're in Native mode. If you see "Entities" and "Kinds", you're in Datastore mode (which won't work — you'd need to recreate).

Issue 3: Interleaved Output Access

See Section 7 for a verification test script.

---

6. Credentials & Config Reference

Environment Variables for Local Development

Create a `.env` file (never commit this):

```bash
```

```
# .env — LOCAL DEVELOPMENT ONLY

# GCP Project
GCP_PROJECT_ID=dreamscape-hackathon
GCP_REGION=us-central1

# Gemini API (Google AI Studio)
GOOGLE_API_KEY=AIzaSyC0QlN_yWtph0CnkwABggBaZFO9A7X7kzc

# Firebase (frontend)
NEXT_PUBLIC_FIREBASE_API_KEY=AIzaSyD6qfA1PfrF5PZx7qSS0k00PYnn9vll034
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=dreamscape-hackathon.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=dreamscape-hackathon
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=dreamscape-hackathon.firebasestorage.app
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=411432678431
NEXT_PUBLIC_FIREBASE_APP_ID=1:411432678431:web:15a0c7d640aa7fcf97d12a
NEXT_PUBLIC_FIREBASE_MEASUREMENT_ID=G-JXMQE6ZBVY

# Cloud Storage
GCS_BUCKET_NAME=dreamscape-media

# Authentication: No service account key file needed!
# Local dev: run "gcloud auth application-default login"
# Cloud Run: service account attached at deploy time
```

.gitignore entries (CRITICAL)

```
.env
*.json          # safety net in case any key files end up here
node_modules/
__pycache__/
venv/
.next/
```

> **Note:** Since we're using Application Default Credentials (no key file), the risk of leaking a service account key is eliminated.

---

7. Gemini Interleaved Output Verification

This is critical — interleaved output (text + images in one response) is the **core category requirement**. Run this test before building anything.

Verification Script

Create this file as `test_interleaved.py` and run it:

```python
```

```python
"""
Test: Verify Gemini interleaved output (text + image) works with your API key.
Run: pip install google-genai Pillow && python test_interleaved.py
"""
import os
from google import genai
from google.genai import types
from PIL import Image
from io import BytesIO

# --- Config ---
API_KEY = "AIzaSyC0QlN_yWtph0CnkwABggBaZFO9A7X7kzc"  # Your AI Studio key

client = genai.Client(api_key=API_KEY)

# --- Test: Interleaved output (text + image) ---
print("Testing interleaved output (text + image in one response)...")
print("-" * 60)

try:
    response = client.models.generate_content(
        model="gemini-2.0-flash-exp",
        contents="Describe a surreal dreamscape scene where a giant clock melts "
                 "over a floating island, then generate an image of this scene.",
        config=types.GenerateContentConfig(
            response_modalities=["TEXT", "IMAGE"],
            temperature=0.9,
        ),
    )

    text_parts = 0
    image_parts = 0

    for part in response.candidates[0].content.parts:
        if part.text:
            text_parts += 1
            print(f"[TEXT PART {text_parts}]")
            print(part.text[:200])
            print()
        elif part.inline_data:
            image_parts += 1
            print(f"[IMAGE PART {image_parts}] mime_type: {part.inline_data.mime_type}")
            # Save the image to verify
            img = Image.open(BytesIO(part.inline_data.data))
            img.save(f"test_dream_scene_{image_parts}.png")
            print(f"  -> Saved as test_dream_scene_{image_parts}.png ({img.size[0]}x{img.size[1]})")
```

```
        print()

    print("-" * 60)
    print(f"RESULTS: {text_parts} text parts, {image_parts} image parts")

    if text_parts > 0 and image_parts > 0:
        print("✅ INTERLEAVED OUTPUT WORKS — You are good to go!")
    elif text_parts > 0 and image_parts == 0:
        print("⚠️  TEXT ONLY — Image generation not available on this model/key.")
        print("   Try model='gemini-2.0-flash-exp' or check Imagen access.")
    else:
        print("❌ UNEXPECTED — Check the raw response above.")

except Exception as e:
    print(f"❌ ERROR: {e}")
    print()
    print("Troubleshooting:")
    print("  1. Is your API key valid? Test at aistudio.google.com")
    print("  2. Is 'gemini-2.0-flash-exp' available? Try 'gemini-2.0-flash'")
    print("  3. Check if interleaved output is enabled for your region")
```

Expected Output

If everything works:

```
[TEXT PART 1]
In this surreal dreamscape, a massive ornate clock...

[IMAGE PART 1] mime_type: image/png
  -> Saved as test_dream_scene_1.png (1024x1024)

RESULTS: 1 text parts, 1 image parts
✅ INTERLEAVED OUTPUT WORKS — You are good to go!
```

If It Fails

Error	Fix
Model not found	Try `gemini-2.0-flash` instead of `gemini-2.0-flash-exp`
Text only, no images	Image generation may not be enabled. Check AI Studio for Imagen access
403 / Permission denied	API key may not have Generative Language API enabled
Region not supported	Interleaved output may be limited to certain regions

Run this test FIRST. Everything else depends on it.

---

8. Architecture Overview

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                              FRONTEND                                       │
│                    Next.js 14 + React + TailwindCSS                         │
│                  Hosted on: Firebase Hosting / Cloud Run                    │
│                                                                             │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │ Dream        │  │ Generation   │  │ Dream        │  │ Analysis     │   │
│  │ Input        │  │ Progress     │  │ Film         │  │ Dashboard    │   │
│  │ Screen       │  │ Screen       │  │ Player       │  │              │   │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘   │
└─────────────────────────────────────────────────────────────────────────────┘
                                │
                                │ REST + WebSocket
                                ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│                          BACKEND (Cloud Run)                                │
│                        Python 3.12 + FastAPI                                │
│                                                                             │
│  ┌─────────────────────────────────────────────────────────────────────┐    │
│  │                        ADK Agent Orchestrator                       │    │
│  │                                                                     │    │
│  │  ┌──────────────────┐    ┌──────────────────────┐                  │    │
│  │  │ Dream            │──▶ │ Visual Director      │                  │    │
│  │  │ Interpreter      │    │ (INTERLEAVED         │                  │    │
│  │  │ (Gemini)         │    │ OUTPUT)              │                  │    │
│  │  └──────────────────┘    └──────────┬───────────┘                  │    │
│  │                                      │                              │    │
│  │                                      ▼                              │    │
│  │  ┌──────────────────┐    ┌──────────────────────┐                  │    │
│  │  │ Narrative        │    │ Scene                │                  │    │
│  │  │ Voice Agent      │──▶ │ Composer             │                  │    │
│  │  │ (Cloud TTS)      │    │ (FFmpeg)             │                  │    │
│  │  └──────────────────┘    └──────────┬───────────┘                  │    │
│  │                                      │                              │    │
│  │                                      ▼                              │    │
│  │  ┌──────────────────┐                                               │    │
│  │  │ Dream            │◀──────────────────────────                    │    │
│  │  │ Analyst          │  (runs async after completion)                │    │
│  │  │ (Gemini)         │                                               │    │
│  │  └──────────────────┘                                               │    │
│  └─────────────────────────────────────────────────────────────────────┘    │
└─────────────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│                    GOOGLE CLOUD SERVICES                     │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ Firestore    │  │ Cloud Storage│  │ Cloud Text-to-│      │
│  │ (dream data, │  │ (images,     │  │ Speech (narration│   │
│  │ themes,      │  │ audio,       │  │ voiceover)   │       │
│  │ analysis)    │  │ videos)      │  │              │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ Artifact     │  │ Cloud Build  │  │ Secret Manager│      │
│  │ Registry     │  │ (CI/CD)      │  │ (API keys)   │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
│  ┌──────────────┐  ┌──────────────────────────────┐         │
│  │ Firebase     │  │ Gemini 2.0 Flash             │         │
│  │ Auth         │  │ (Interleaved text + image)   │         │
│  │ (Google SSO) │  │ via Google GenAI SDK         │         │
│  └──────────────┘  └──────────────────────────────┘         │
└──────────────────────────────────────────────────────────────┘
```

---

9. Technology Stack

Layer	Technology	Purpose
AI Model	Gemini 2.0 Flash	Interleaved multimodal output, dream interpretation, theme analysis
Image Gen	Gemini native image generation (Imagen 3 under the hood)	Surrealist dream scene generation
Agent Framework	Google ADK (Agent Development Kit)	Multi-agent orchestration
SDK	Google GenAI SDK (Python)	Direct Gemini API calls with interleaved output
Backend	Python 3.12 + FastAPI	REST API, WebSocket streaming, async pipeline
Frontend	Next.js 14 + React + TailwindCSS	Dream input, film player, journal, analysis
Video	FFmpeg + MoviePy	Ken Burns effect, text overlays, audio mixing
Voice	Google Cloud Text-to-Speech	Ethereal narration voiceover
Database	Cloud Firestore (Native mode)	Dream entries, themes, analysis data

Layer	Technology	Purpose
Storage	Cloud Storage (`dreamscape-media`)	Images, audio, video files
Hosting	Cloud Run (backend) + Firebase Hosting (frontend)	Containerized services
Containers	Docker + Artifact Registry	Container image management
IaC	Terraform	Infrastructure automation (bonus)
Auth	Firebase Authentication (Google Sign-In)	User identity

---

10. Agent Pipeline Design

Pipeline Flow

```
User Input (messy dream text)
    │
    ▼
┌──────────────────────────────┐
│ Dream Interpreter  │ Gemini 2.0 Flash
│ Agent              │ Extracts scenes, emotions, symbols
│                    │ Output: DreamSchema JSON
└──────────────────────────────┘
    ▼
┌──────────────────────────────┐
│ Visual Director    │ Gemini 2.0 Flash (INTERLEAVED OUTPUT)
│ Agent              │ response_modalities: ["TEXT", "IMAGE"]
│                    │ Output: Narration text + scene images
└──────────────────────────────┘
    ▼
┌──────────────────────────────┐
│ Narrative Voice    │ Google Cloud Text-to-Speech
│ Agent              │ WaveNet voice, slow rate, low pitch
│                    │ Output: WAV audio files
└──────────────────────────────┘
    ▼
┌──────────────────────────────┐
│ Scene Composer     │ FFmpeg + MoviePy
│ Agent              │ Ken Burns, dissolves, text overlays
│                    │ Output: Final MP4 video
└──────────────────────────────┘
    ▼
┌──────────────────────────────┐
```

```
│ Dream Analyst      │ Gemini 2.0 Flash (runs async)
│ Agent              │ Cross-dream pattern recognition
│                    │ Output: ThemeReport JSON
└──────────────────────────────────────────────────
```

Agent Configurations

Agent	Model	Temperature	Key Config
Dream Interpreter	gemini-2.0-flash	0.7	Structured JSON output
Visual Director	gemini-2.0-flash-exp	0.9	`response_modalities: ["TEXT", "IMAGE"]`
Narrative Voice	Cloud TTS (WaveNet)	N/A	Pitch: -2.0, Rate: 0.75x
Scene Composer	N/A (FFmpeg)	N/A	Ken Burns, 30-90s output
Dream Analyst	gemini-2.0-flash	0.5	Lower temp for analysis accuracy

---

11. Data Models

DreamEntry (Firestore: `dreams` collection)

```json
```

```
{
  "id": "auto-generated-uuid",
  "userId": "firebase-auth-uid",
  "rawText": "I was falling through a library where books had wings...",
  "createdAt": "2026-03-01T08:30:00Z",
  "dreamSchema": { },
  "generatedAssets": {
    "sceneImages": ["gs://dreamscape-media/dreams/{id}/scene_1.png", "..."],
    "narrationAudio": "gs://dreamscape-media/dreams/{id}/narration.wav",
    "ambientAudio": "gs://dreamscape-media/dreams/{id}/ambient.mp3",
    "finalVideo": "gs://dreamscape-media/dreams/{id}/dream_film.mp4"
  },
  "analysis": {
    "emotions": ["wonder", "anxiety", "curiosity"],
    "symbols": ["falling", "books", "wings", "library"],
    "title": "The Library of Winged Pages",
    "mood": "surreal"
  },
  "status": "processing | complete | failed"
}
```

DreamSchema (generated by Interpreter Agent)

```
json
```

```
{
  "title": "The Library of Winged Pages",
  "scenes": [
    {
      "description": "Falling through infinite vertical space lined with bookshelves",
      "entities": ["dreamer (falling)", "bookshelves", "void"],
      "emotion": "vertigo mixed with wonder",
      "visualStyle": "dark with warm golden light from book spines, motion blur",
      "transitionTo": "dissolve"
    },
    {
      "description": "Books detach from shelves and sprout feathered wings, circling the dreamer",
      "entities": ["winged books", "feathers", "dreamer"],
      "emotion": "awe and gentle fear",
      "visualStyle": "soft focus, white feathers catching light, dreamlike glow",
      "transitionTo": "morph"
    }
  ],
  "overallMood": "surreal",
  "symbols": [
    { "name": "falling", "possibleMeaning": "loss of control or surrender", "frequency": 1 },
    { "name": "books", "possibleMeaning": "knowledge, memory, stories", "frequency": 1 },
    { "name": "wings", "possibleMeaning": "freedom, escape, transformation", "frequency": 1 }
  ],
  "narrativeArc": "A descent through knowledge becoming alive — from passive observer to surrounded participant",
  "colorPalette": ["#1a1a2e", "#d4a843", "#f5f5dc", "#ffffff"]
}
```


ThemeReport (generated by Analyst Agent)

```
json
```

```
{
  "userId": "firebase-auth-uid",
  "analyzedAt": "2026-03-15T12:00:00Z",
  "totalDreams": 12,
  "recurringSymbols": [
    {
      "symbol": "water",
      "count": 7,
      "dreamIds": ["id1", "id3", "id5", "..."],
      "interpretation": "Water appears in 58% of your dreams, most often during anxious dreams. In dream psychology, water o
    }
  ],
  "emotionalPatterns": [
    {
      "pattern": "Anxiety peaks on Sunday nights",
      "frequency": "4 of 5 Sunday dreams",
      "correlation": "Likely anticipation of the work week"
    }
  ],
  "connections": [
    {
      "dreamId1": "id3",
      "dreamId2": "id8",
      "sharedElements": ["endless hallway", "locked door"],
      "insight": "Both dreams feature themes of inaccessibility and searching"
    }
  ]
}
```


---

12. API Design

REST Endpoints

Method	Endpoint	Description
POST	`/api/dreams`	Submit a new dream; triggers full generation pipeline
GET	`/api/dreams`	List all dream entries (paginated)
GET	`/api/dreams/:id`	Get single dream with all assets
GET	`/api/dreams/:id/status`	Poll generation status
GET	`/api/dreams/:id/video`	Get signed URL for dream film

Method	Endpoint	Description
GET	`/api/analysis`	Get latest ThemeReport
POST	`/api/analysis/refresh`	Trigger fresh journal analysis
DELETE	`/api/dreams/:id`	Delete dream entry and assets

WebSocket

`ws://api/dreams/:id/stream` — Real-time progress events:

```
→ { "event": "interpretation_complete", "data": { dreamSchema } }
→ { "event": "scene_image_generated", "data": { sceneIndex, imageUrl } }
→ { "event": "narration_ready", "data": { audioUrl } }
→ { "event": "video_complete", "data": { videoUrl } }
```

---

13. Frontend Design

Design Philosophy

Dark theme by default (nighttime/sleep aesthetic). Soft purple and indigo accents. Slow, dreamy animations. No sharp transitions. The feel: calm, mysterious, personal.

Key Screens

Screen	Purpose
Dream Input	Full-screen textarea with star-field animation, voice input button
Generation Progress	Pipeline visualization, scene images fading in one by one
Dream Film Player	Custom video player with scene navigation, text overlay toggle
Dream Journal Gallery	Grid of thumbnails, filterable by date/emotion/symbol
Analysis Dashboard	Recurring symbol constellation, emotional timeline, AI insights

---

14. Repository Structure

```
dreamscape/
├── README.md                # Spin-up instructions + architecture diagram
├── ARCHITECTURE.md          # Detailed architecture docs
```

```
├── deploy.sh                  # One-command deployment script
├── docker-compose.yml         # Local development
├── .env.example               # Template (no real secrets)
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py                # FastAPI entry point
│   ├── agents/
│   │   ├── dream_interpreter.py   # Gemini dream parsing (ADK)
│   │   ├── visual_director.py     # Interleaved output agent
│   │   ├── narrative_voice.py     # Cloud TTS narration
│   │   ├── scene_composer.py      # FFmpeg video composition
│   │   └── dream_analyst.py       # Recurring theme analysis
│   ├── models/                # Pydantic schemas
│   ├── services/              # Firestore, GCS, TTS wrappers
│   ├── api/                   # FastAPI routers
│   └── tests/
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── src/
│   │   ├── app/               # Next.js app router
│   │   ├── components/        # React components
│   │   ├── hooks/             # WebSocket, auth hooks
│   │   └── lib/               # API client, Firebase config
│   └── public/
├── infra/
│   ├── main.tf                # Terraform main
│   ├── variables.tf
│   ├── outputs.tf
│   └── modules/               # Cloud Run, Firestore, GCS
└── docs/
    ├── architecture-diagram.png
    └── demo-video-script.md
```

---

15. Development Roadmap

Phase	Days	Focus	Deliverables
Phase 1	1–3	Infrastructure	GCP setup, Terraform, Docker, FastAPI + Next.js boilerplate
Phase 2	4–7	Core AI	Dream Interpreter Agent, DreamSchema, Firestore integration
Phase 3	8–12	Multimodal Gen	Visual Director (interleaved output), Cloud TTS narration

Phase	Days	Focus	Deliverables
Phase 4	13–16	Video Pipeline	Scene Composer (FFmpeg), Ken Burns, audio mixing, MP4
Phase 5	17–20	Frontend	Journal gallery, film player, WebSocket streaming
Phase 6	21–24	Analysis	Dream Analyst Agent, symbol tracking, analysis dashboard
Phase 7	25–28	Submission	Testing, demo video, GCP proof, blog post, polish

---

16. Demo Script (4-Minute Video)

Time	Segment	Content
0:00–0:20	**HOOK**	Pre-generated dream film playing. "What if you could watch your dreams?"
0:20–0:50	**PROBLEM**	Dreams are ephemeral (90% forgotten in 10 min). Text journals are flat.
0:50–1:40	**LIVE DEMO 1**	Type messy dream → hit generate → watch progressive loading
1:40–2:20	**LIVE DEMO 2**	Play completed dream film — visuals, narration, transitions
2:20–2:50	**JOURNAL + ANALYSIS**	Tour gallery, show recurring symbols and patterns
2:50–3:20	**TECH DIVE**	Architecture, Gemini interleaved output, ADK, GCP console
3:20–3:50	**VALUE**	Personal insight, creative inspiration, therapeutic potential
3:50–4:00	**CLOSE**	"Your subconscious, visualized." URL + GitHub

---

17. Submission Checklist

#	Item	Type	Status
1	Text description of features/tech/learnings	Mandatory	☐
2	Public GitHub repository URL	Mandatory	☐
3	README with spin-up instructions	Mandatory	☐
4	GCP deployment proof recording	Mandatory	☐
5	Architecture diagram (in image carousel)	Mandatory	☐
6	Demo video (<4 min, no mockups)	Mandatory	☐

#	Item	Type	Status
7	Blog post with #GeminiLiveAgentChallenge	Bonus	☐
8	IaC scripts in /infra (Terraform)	Bonus	☐
9	GDG profile link	Bonus	☐

18. Local Development Setup

Prerequisites

- Python 3.12+
- Node.js 20+
- Docker Desktop
- FFmpeg (`brew install ffmpeg` / `apt install ffmpeg`)
- gcloud CLI (authenticated)

Quick Start

```bash
```

```
# 1. Clone the repo
git clone https://github.com/YOUR_USERNAME/dreamscape.git
cd dreamscape

# 2. Authenticate with GCP (this replaces the need for any service account key)
gcloud auth login
gcloud config set project dreamscape-hackathon
gcloud auth application-default login   # <-- THIS is your "key" for local dev

# 3. Copy environment file
cp .env.example .env
# Edit .env with your Gemini API key and Firebase config

# 4. Backend
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000

# 5. Frontend (new terminal)
cd frontend
npm install
npm run dev

# 6. Open http://localhost:3000
```

Run with Docker Compose

```bash
docker-compose up --build
# Backend: http://localhost:8000
# Frontend: http://localhost:3000
```

Next Step: Run the interleaved output verification test (Section 7) to confirm Gemini is ready, then we start building the backend agents.